Assignment 5: Quality Attributes and Architecture

Learning Objectives:

Continue practising Scrum activities

Get better at using the project management tool

Document your development processes

Document your quality requirements.

Document your architectural design.

Release MVP v2 and ship it to the customer.

ToDo:

SPRINT 5

- Groom your backlog, update the issues.
- Plan the sprint (in the beginning of the week) and create a milestone;
- Review the sprint (at the end of the week).
- Run a retrospective on the process (at the end of the week).
- Report in Sprint tracking table.

GITLAB/GITHUB

- Create three issue (form) templates (one for each of the following issue variants):
  - User story;
    - The template must require acceptance criteria in the GIVEN/WHEN/THEN format.
  - Bug report;
    - The template must require reproduction steps, expected behavior, and actual behavior.
  - Technical task.
    - The template must require a subtasks checklist or links to subtask issues.
- Create one pull request template. The template must require a link to the issue that this pull request addresses.
- Extend your automated testing suite.
  - Implement 5 more unit tests covering the most critical parts of your code.
  - Implement 5 more integration tests.
- Improve your CI pipeline:
  - Integrate another static analysis tool (other than linter).
  - Make sure your old and new tests run and fail when the code breaks.
- (Optional for extra points) Add continuous delivery (CD) to your pipeline.

README – DEVELOPMENT SECTION:

Kommentiert [1]: Ideally, you should update descriptions of sprint issues using an LLM (e.g., Gemini in Google AI Studio). For each issue, ask the LLM to rewrite the issue description as if the issue was created using the corresponding template (see later tasks).

However, for the assignment, it's enough if you just create all new issues using issue templates.

Kommentiert [2]: See:

https://github.com/inno-swp-2025/we-have-tiktok-at-home?tab=readme-ov-file#issue-form-templates

https://github.com/inno-swp-2025/we-have-tiktok-at-home?tab=readme-ov-file#templates

Kommentiert [3]: See:

https://github.com/inno-swp-2025/we-have-tiktok-at-home?tab=readme-ov-file#pull-requests

https://github.com/inno-swp-2025/we-have-tiktok-at-home?tab=readme-ov-file#templates

Kommentiert [4]: It's OK to have CI and CD in the same workflow file. Split them into stages or jobs if necessary.

- In the "## Development" section of the README.md file in the repository root directory, document the development policies that you used during sprint 5.
    - In the "### Kanban board" subsection, provide a link to the board and for each column in the kanban board, document entry criteria for each column.
        - Update your kanban board(s) in accordance with what you described.
    - In the "### Git workflow" subsection, document the entire git workflow.
        - Specify which base workflow you adapted to your context (e.g., GitHub flow, Gitflow, etc).
        - Explain / Define rules for:
            - Creating issues from the defined templates (link the earlier defined templates);
            - Labelling issues;
            - Assigning issues to team members;
            - Creating, naming, merging branches;
            - Commit messages format;
            - Creating a pull request for an issue using a pull request template (link the earlier defined pull request template);
            - Code reviews;
            - Merging pull requests;
            - Resolving issues.
        - Illustrate your git workflow using a Gitgraph diagram.
    - In the "### Secrets management" subsection, document your rules for secrets management (e.g. passwords or API keys). Mention where you store your secrets without revealing sensitive information.

QUALITY

- Document your quality characteristics (a.k.a quality attributes):
    - Read through ISO model 25010 with the whole team.
    - Pick at least 3 quality sub-characteristics important to your customer and confirm them with the customer.
    - In the "docs/quality-assurance/quality-attribute-scenarios.md" file, group chosen sub-characteristics in "## <characteristic name>" sections and for each chosen sub-characteristic, in a subsubsection "### <sub-characteristic name>":
        - Explain why that sub-characteristic is important to your customer.
        - Provide tests for that sub-characteristic in Quality Attribute Scenario format.
        - For each test, specify how exactly you are going to execute it.
    - In the "## Quality assurance" section of the README.md, in the "### Quality attribute scenarios", provide a link to the "docs/quality-assurance/quality-attribute-scenarios.md" file.
- In the "## Quality assurance" section of the README.md, document your quality assurance automation.
    - In the "### Automated tests" subsection, document:

Kommentiert [5]: If you have multiple kanban boards like in the demo project:
- name this section "Kanban boards".
- provide documentation for each board in a subsubsection.

Kommentiert [6]: Suggested board structure:
- To Do
- In Progress
- In Review
- Ready to deploy
- User Testing
- Done

Some of the entry criteria can include
- Issues is estimated.
- Issue is formatted using the template.
- A new branch is created for each issue when starting to work on it
- MR is created
- Reviewer assigned.
- Review is complete
- MR approved

Kommentiert [7]: Q: Should we specify a Definition of Done (DoD) if we have entry criteria for each stage?

A: There may be a DoD that covers stages on a high level. What matters though is that everybody is clear what the workflow is and what Done means.

Kommentiert [8]: Entry criteria for a column specify conditions that must all be satisfied for an issue to be moved into that column from a column on the left.

In the demo project, criteria are written as column descriptions so that they're always visible. It's assume

Kommentiert [9]: the idea here is that you all need to be on the same page. and if a new person were to join your team, you'd tell him go read this section, this is how we do things.

Kommentiert [10]: Tip:
Use Mermaid.

https://docs.mermaidchart.com/mermaid-oss/syntax/gitgraph.html

Kommentiert [11]: Suggested approach:
- Store keys for encrypting and decrypting secrets in a secure secrets/password manager.
- Store encrypted secrets in the repository separately from code.
- Pass them to processes as environment variables

Kommentiert [12]: See:

https://www.iso25000.com/index.php/en/iso-25000-standards/iso-25010

Kommentiert [13]: and an LLM.

Kommentiert [14]: See:

https://www.cs.unibo.it/cianca/wwwpages/as/5.pdf#page=26

https://greg4cr.github.io/courses/spring21dit635/Lectu

- Which tools you used for testing.
- Which types of tests you implemented.
- Where tests of each type are in the repository.

BUILD AND DEPLOYMENT

- In the "## Build and deployment" section of the README.md, document your CI (or, CI/CD).
  - In the "### Continuous Integration" subsection, document your CI pipeline.
    - For each CI workflow file:
      - Provide a link to that file.
      - Provide a list of static analysis tools and testing tools that you use in the CI part. For each tool in the list, briefly explain what you use it for.
    - Provide a link to where all CI workflow runs can be seen.
  - If you added CD, in the "### Continuous Deployment" subsection, document your CD pipeline similarly to your CI pipeline.

ARCHITECTURE

- Document your architecture:
  - In the repository root directory, create the "docs/architecture" directory. In "docs/architecture", create a subdirectory for each README.md subsection mentioned below (e.g., "static-view", "dynamic-view", "deployment-view"). In each subdirectory, put files (e.g., PlantUML code, rendered diagram images) relevant to the corresponding subsection and reference all these files when writing that subsection.
  - In the "## Architecture" section of the README.md:
    - In the "### Static view" subsection, document the static view on your architecture using UML Component diagram, comment on the coupling and cohesion of your codebase, and discuss how your design decisions affect the maintainability of your product.
    - In the "### Dynamic view" subsection, document the dynamic view of your architecture using UML Sequence diagram for a non-trivial request that showcases your system. The request must involve several components and multiple transactions between the components.
      - Test and report how much time this scenario takes to execute in your production environment.
    - In the "### Deployment view" subsection, document the deployment view of your architecture (can be a custom view with a legend), comment on the deployment choices and how it is to be deployed on the customer's side.

PRODUCT

Kommentiert [15]: Suggested tools:

https://plantuml.com/component-diagram

Required/provided interfaces:
https://stackoverflow.com/a/57134601

Kommentiert [16]: As per https://iso25000.com/index.php/en/iso-25000-standards/iso-25010. Explain with respect to maintainability sub-characteristics.

Kommentiert [17]: Suggested tools:

https://plantuml.com/sequence-diagram
https://mermaid.js.org/syntax/sequenceDiagram.html

Kommentiert [18]: Suggested tools:

https://www.lucidchart.com/pages/uml-deployment-diagram

https://plantuml.com/deployment-diagram

https://likec4.dev/ (diagram can be hosted on GitHub Pages)

https://app.diagrams.net/ (diagram can be stored in the repo and edited in VS Code via the hediet.vscode-drawio extension)

Not very convenient, but worth mentioning:

https://structurizr.com/

- MVP v2 (targets feedback from MVP v1, adds some new functionality)
    - Increase value by addressing feedback: fix bugs, improve UX, and add new functionality.
    - Record a demo video (up to 2 mins) showcasing the current state of the product and highlighting what you fixed and what's new.
    - Release MVP v2 and any launch/access instructions (if TA cannot launch/access and the problem is on your side, you get 0).
    - Define 2 new user acceptance tests with the customer for the newly implemented functionality based on the acceptance criteria.
    - In the "docs/quality-assurance/user-acceptance-tests.md" file, in separate sections, write all acceptance tests (old and new). For each test, provide links to related acceptance criteria.
    - In the "## Quality assurance" section of the README.md, in the "### User acceptance tests" section, provide a link to the "docs/quality-assurance/user-acceptance-tests.md" file.
    - Record the customer going through the old and new acceptance tests while commenting out loud.
    - Report the results of user testing:
        - Which of the acceptance tests you passed (both fixed and new).
        - What still needs to be fixed in the product.
        - What most important feedback points you received.
    - Update your product roadmap using the template.
- In the "## Usage" section of the README.md, explain how to use your MVP v2. Provide information for access if needed, for example authentication credentials and so on. Make sure the customer and TA can launch/access and inspect your product after following the instructions in that section.

Deliverables:

Submit a PDF containing the following:

- Title page:
    - Project, customer, and team members (full name, Innopolis email, GitHub/GitLab username)
    - Who did what during this sprint
    - Who **DID NOT** participate in any of the activities

- Link to your product backlog (issues in the repo or a specific kanban board).
- Link to the Sprint 5 milestone with the sprint goal, completed issues, and corresponding pull requests;
- The Sprint tracking table (based on the updated template);

- The updated product roadmap using the provided template showing:
    - The phases of your project, where you are right now, and what are the future plans.
    - Link to the deployed MVP v2;
    - Link to necessary launch/access instructions.
    - Link to the demo video of the MVP2;

    - Link to the recording of the Usability testing session.
- Report on the main findings from the testing session, including the performance result (execution time) for the dynamic view scenario.
- Retrospective on the process and customer collaboration.

- Links to specific pull request(s) or commit(s) where the 10 new tests (5 unit, 5 integration) were added.
- Link to your CI workflow run which contains results of running analysis tools and all tests.
- Link to your CD workflow run if you implemented CD.

- Link to the README.md with "Usage", "Architecture", "Development", "Quality assurance", "Build and deployment" sections (in this order) having content specified in the ToDo section of this assignment.

- Links to two issues created and resolved according to the defined git workflow procedure.
- Links to two pull requests that were created, reviewed, and merged in accordance with the defined git workflow procedure.

- Report on AI usage.

MAKE SURE TO KEEP YOUR DOCS CONCISE AND TO THE POINT. MEANINGLESS LOW-QUALITY TEXT MIGHT LEAD TO PENALTIES.

IF THE TA CAN'T ACCESS A MATERIAL BY THE LINK THAT YOU PROVIDE, THE TA WILL GIVE 0 FOR THE CORRESPONDING PART OF THE ASSIGNMENT.

How to test links:
- Test in incognito mode.
- Use different accounts, browsers, platforms.
- Ask coursemates to check.
- If still unsure, ask the TA to check.